Episodio 8 de 14: Latencia y memoria retenida

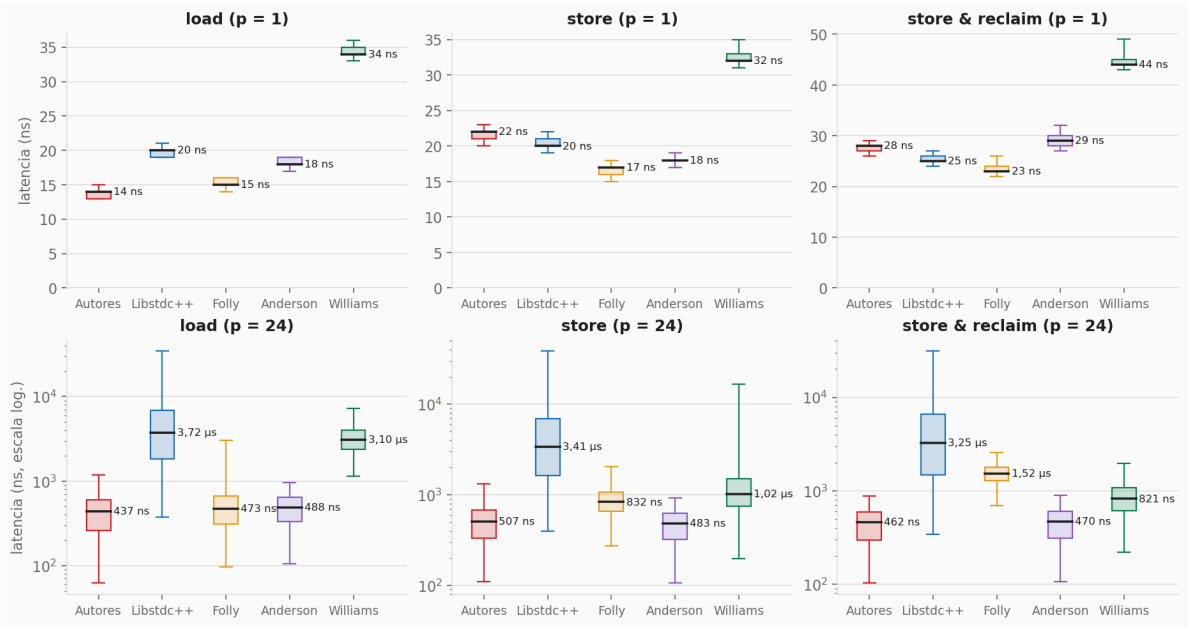
Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 7, «Rendimiento: quién gana, y el misterio de Folly».
- **Episodio siguiente:** episodio 9, «¿Es correcto? Detectores y la cota wait-free».

Lo esencial de este episodio:

- El load de Autores es el de menor latencia en los tres casos, aunque con contención gana por poco.
- Con contención, la latencia cambia mucho de una ronda a otra. Con 15 rondas, la afirmación 11 del paper dejó de confirmarse.
- El programa oficial de latencia se compila sin -mcx16, y así Williams lleva un candado escondido; corregido eso, con contención no es el peor. Anderson retiene hasta 19 960 bloques (469 KB) con 24 hilos; los demás no pasan del límite.

6.2 Latencia: la figura 6



Latencia de cada operación con y sin contención

Cómo leerla. Cada caja resume muchísimas repeticiones de una operación. La raya negra es la **mediana**; la caja contiene la mitad central de las medidas; los bigotes llegan hasta el 1 % más rápido y el 1 % más lento. Arriba, sin competencia (p = 1), en escala normal. Abajo, con 23 hilos molestando (p = 24), en escala logarítmica, donde cada marca vale 10 veces más que la anterior.

Las medianas, en nanosegundos (en negrita, la mejor de cada fila; la tabla completa con el paper está en `tablas.md`):

Operación	Autores	Anderson	Folly	Williams	Libstdc++
`load`, p = 1	**14**	18	15	34	20

Operación	Autores	Anderson	Folly	Williams	Libstdc++
`load`, p = 24	**437**	488	473	3 104	3 716
`load`, p = 56	**1 502**	1 540	2 977	23 738	61 687
`store`, p = 1	22	18	**17**	32	20
`store`, p = 24	507	**483**	832	1 021	3 411
`store`, p = 56	1 516	**1 498**	4 813	4 105	52 832
`store & reclaim`, p = 1	28	29	**23**	44	25
`store & reclaim`, p = 24	**462**	470	1 521	821	3 248
`store & reclaim`, p = 56	**813**	1 499	6 525	4 439	47 198

`load`: se cumple lo que promete el paper. Autores tiene la mediana más baja en los tres casos. Eso sí, con contención el margen es pequeño: entre un 8 % y un 12 % frente a Folly y Anderson con p = 24, y un empate con Anderson con p = 56, porque los rangos de las rondas se solapan.

`store` sin contención: aquí no se cumple. El paper dice que Autores es siempre el segundo, tras Anderson. Aquí queda por detrás de Folly, Anderson y Libstdc++. Son 5 ns de diferencia, un puñado de ciclos, pero el orden es otro.

`store` con contención: empate. El paper dice que Autores vuelve a ser el mejor. Aquí empata con Anderson (507 frente a 483 ns y 1 516 frente a 1 498 ns), y ambos quedan por delante del resto.

`store & reclaim`: se cumple. Con p = 1, los cinco son más lentos que su `store` normal, porque destruir cuesta. Con contención, algunos van **más rápido**: Autores baja de 1 516 a 813 ns con p = 56. El paper lo atribuye a que destruir hace de pausa y alivia el atasco. Esa explicación no se comprobó aquí. Y ojo: sin Docker y con 15 rondas, esa mejora casi desaparece (ver «Sin Docker y con 15 rondas», más abajo).

¿Tenía razón el paper al omitir a Williams? Solo en parte. El paper dice que «su latencia casi siempre era mayor». Sin contención, sí: es el peor en las tres operaciones, con 32 a 44 ns. Con contención, no: en los 6 casos supera a Libstdc++ (en `load` con p = 24, por poco) y en 3 de 6 supera también a Folly. Sigue por detrás de Autores y de Anderson: su `load` con p = 24 tarda 3,1 µs, 7 veces más que Autores. Sin Docker, con 15 rondas, el cuadro se parece, pero algo más débil (ver más abajo).

Una trampa que casi se cuela: el candado escondido. La primera medida de Williams decía que ese mismo `load` tardaba 40,7 µs, 93 veces más que Autores. Parecía confirmar la frase del paper. Pero era un **error de medida**, y fue nuestro.

Williams guarda 16 bytes y necesita la instrucción `cmpxchg16b`. El compilador solo la usa si se le pide con la bandera `-mcx16`. El programa de rendimiento la lleva, pero el de latencia **no**: como el paper deja a Williams fuera de esa prueba, ese binario no la necesitaba. Al activar su línea, `boost::atomic` no pudo usar la instrucción y, sin avisar, recurrió a un candado interno. Fue como cronometrar a un corredor con una mochila que nadie le vio poner.

Williams, mediana en ns	Con candado escondido	Sin candado
`load`, p = 1	52	**34**

Williams, mediana en ns	Con candado escondido	Sin candado
`load`, p = 24	40 680	**3 104**
`load`, p = 56	36 820	**23 738**
`store`, p = 24	4 420	**1 021**
`store & reclaim`, p = 24	4 594	**821**

Se descubrió comparando con el programa del equipo, y se confirmó mirando el binario: tenía llamadas a `lock_pool` y ni un solo `cmpxchg16b`. Tras compilarlo con `-mcx16`, 0 llamadas y 10 instrucciones. Las medidas malas se guardaron en `docker/latencia_williams_con_candado.csv`.

¿Afecta a los demás? No: no usan átomos de 16 bytes. Pero queda una duda. El artefacto publicado compila ese binario sin `-mcx16`. Si los autores midieron a Williams así, su frase vendría en parte del candado. **No podemos saberlo:** el paper no publica esos datos.

Las colas: dos historias que la mediana no cuenta. Recuerda: la mediana es la operación típica, y la cola es lo malo que puede ponerse. El percentil 99,95 es el valor que solo supera 1 operación de cada 2 000.

	Autores	Anderson	Folly	Williams	Libstdc++
`store & reclaim`, p = 1: p99	29	32	26	49	27
`store & reclaim`, p = 1: p99,95	30	**7 895**	36	52	34
`load`, p = 24: p99,95	1 634	1 408	5 566	10 019	**65 230**

Historia 1: la caja de pendientes de Anderson. Sin contención, Anderson va tan fino como los demás… hasta el p99,95, donde salta de 32 a casi 8 000 ns. ¿Por qué justo ahí? Anderson vacía su caja cada 2 000 retiradas: 1 de cada 2 000 es el 0,05 %, precisamente lo que mira el p99,95. Una operación de cada 2 000 tiene que recorrer las mesas de todos. La coincidencia es demasiado exacta para ser casualidad, aunque no se midió directamente. Con contención (p = 24) este salto no se distingue, porque el atasco lo tapa.

Historia 2: lo que significa de verdad «lock-free». Con 23 escritores cambiando el tablón sin parar, el `load` de Williams necesita ganar dos bucles de `DWCAS`, y cada cambio le hace perder y volver a empezar. El sistema avanza, porque los escritores sí terminan, pero el lector se queda atrás: 1 lectura de cada 2 000 tarda 10 µs, y la de Autores, 1,6 µs, unas 6 veces menos. Es la puerta giratoria a empujones frente al torniquete propio de la Parte 2.2, medida con cronómetro. Con p = 56 no se puede ver esto: hay más hilos que núcleos y las colas de casi todos llegan a milisegundos.

Un aviso sobre p = 24. Sin fijar los hilos, algunas medidas de ese punto salen en dos grupos: unas cerca de 420 ns y otras cerca de 1 450 ns. Con los hilos fijados, ese segundo grupo no apareció en las 3 rondas de `todos24` (419-427 ns en Autores). Sin Docker se ve lo mismo: 582 a 1 959 ns sueltos, y 802 a 813 ns fijados. Y con los programas oficiales sin Docker, en 15 rondas, el `load` de Autores dio entre 399 y 445 ns en 13 rondas, y 1 265 y 1 595 ns en las otras dos. Son indicios de que depende de dónde coloca Linux al hilo medido, aunque 3 rondas no bastan para afirmarlo. A Williams le pasa más: su `load` va de 1,4 a 13,3 µs según la ronda. Por eso las medianas de p = 24 hay que leerlas con cuidado.

¿Cuánto fiarse de estas medianas? Con 1 hilo, mucho. Con contención, bastante menos. Esta tabla compara, punto a punto, la ronda más alta con la más baja. Es como cronometrar

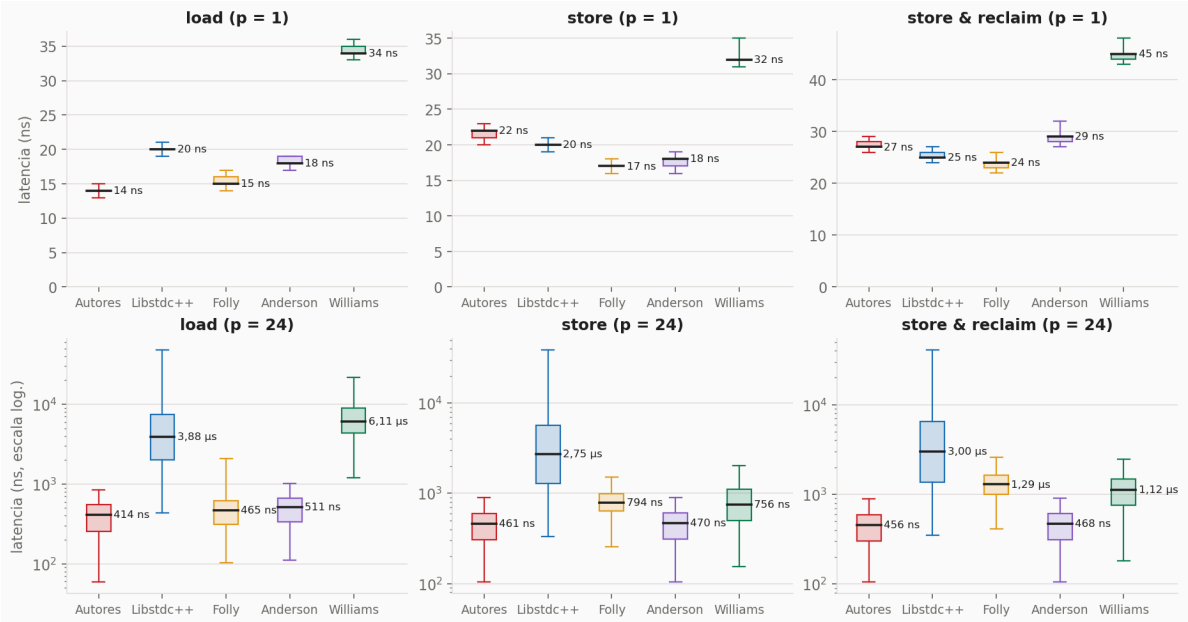
varias veces al mismo corredor: 1,00× significa que el cronómetro siempre marca lo mismo, y 3× que la misma carrera puede dar 10 s o 30 s.

Hilos (p)	Medida	Docker, oficiales (5 rondas)	Sin Docker, oficiales (15 rondas)	Sin Docker, `bench.cpp` (5 rondas)
1	mediana	1,00×	1,00×	1,11×
1	cola (p99)	1,00×	1,03×	1,16×
24	mediana	3,40× (peor caso 13,6×)	5,75× (peor 36,9×)	1,84× (peor 10,4×)
24	cola (p99)	3,33× (peor 28,1×)	6,19× (peor 127,3×)	1,50× (peor 36,0×)
56	mediana	1,81× (peor 4,6×)	7,54× (peor 34,1×)	4,32× (peor 30,3×)
56	cola (p99)	2,36× (peor 4,6×)	10,07× (peor 1 824,2×)	12,91× (peor 339,6×)

Dicho de otra forma: si dos algoritmos se diferencian en un 20 % con p = 24, esa diferencia no significa nada, porque un mismo algoritmo cambia más que eso de una ronda a otra. Solo valen las diferencias enormes, como la de Libstdc++ con los demás.

¿Por qué la columna de 15 rondas sale **peor** que la de 5? Porque compara la ronda más alta con la más baja, y con más rondas hay más ocasiones de pillar una muy buena y una muy mala. Es como tirar un dado: en 15 tiradas es más fácil que salgan un 1 y un 6 que en 5. Más rondas no hacen más estable cada ronda; lo que hacen es que la **mediana** dependa menos de una ronda con mala suerte.

Sin Docker y con 15 rondas: ¿se sostiene? Para salir de dudas, la latencia se repitió con los mismos programas oficiales, fuera de Docker y con 15 rondas en vez de 5. La figura se lee igual que la de arriba:



Latencia sin Docker con los programas oficiales

Las medianas, en nanosegundos (tabla «Latencia: medianas en ns (nativo, binarios oficiales)» de `tablas.md`; en negrita, la mejor de cada fila):

Operación	Autores	Anderson	Folly	Williams	Libstdc++
`load`, p = 1	**14**	18	15	34	20
`load`, p = 24	**414**	511	465	6 106	3 882
`load`, p = 56	**1 473**	1 572	3 281	22 150	40 463
`store`, p = 1	22	18	**17**	32	20
`store`, p = 24	**461**	470	794	756	2 746
`store`, p = 56	**1 531**	1 554	1 534	4 399	44 224
`store & reclaim`, p = 1	27	29	**24**	45	25
`store & reclaim`, p = 24	**456**	468	1 290	1 116	3 002
`store & reclaim`, p = 56	**1 480**	1 487	1 947	5 529	43 235

Con p = 1 todo sale igual que con Docker, casi al nanosegundo. Con contención, fila por fila de la Parte 7:

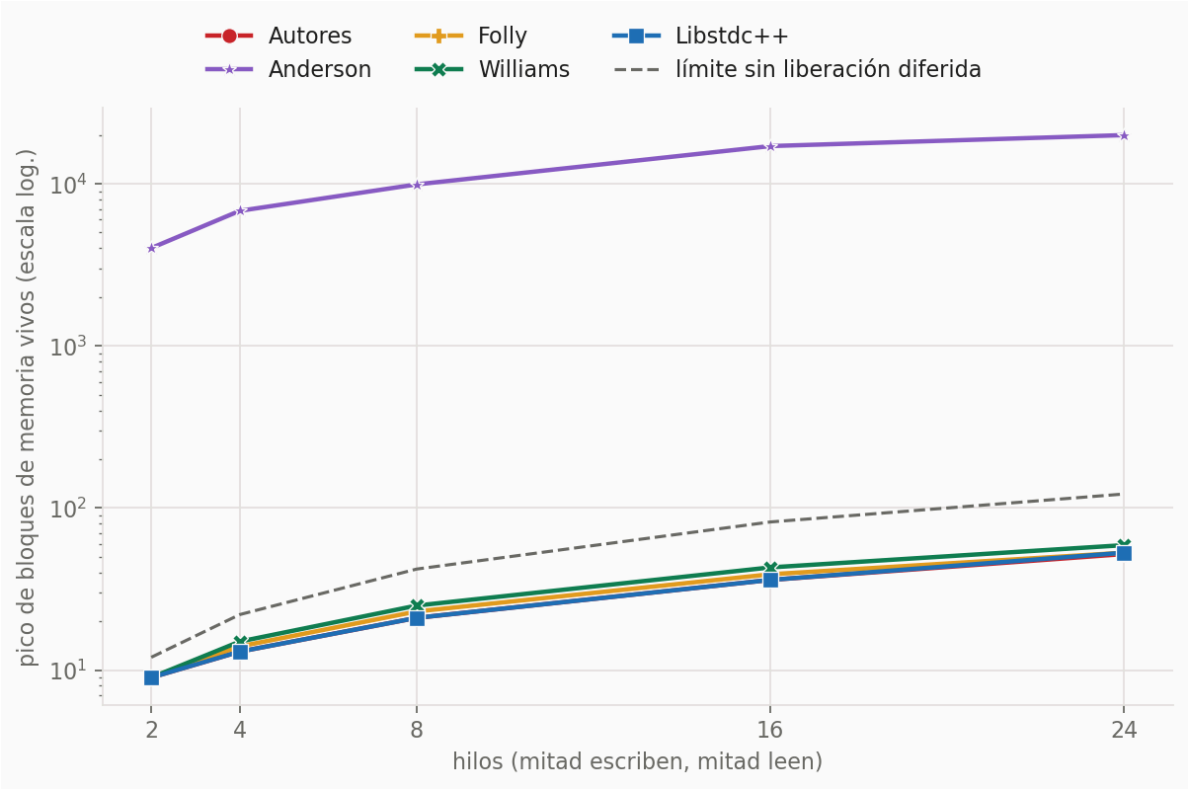
- **Fila 7, el `load`: se sostiene, por poco.** Autores tiene la mediana más baja en los tres casos. Pero con p = 24 las rondas de Autores, Folly y Anderson salen en dos grupos: unas por debajo de 520 ns y otras por encima de 1 000 ns. En el grupo rápido, Autores queda entre 399 y 445 ns; Folly, entre 420 y 517, y Anderson, entre 451 y 511. Autores cae menos veces en el grupo lento (2 de 15 rondas, frente a 6 de Folly y 7 de Anderson), aunque con 15 rondas no se puede afirmar que eso sea una propiedad del algoritmo. Con p = 56, Anderson queda a un 7 % y los rangos se solapan: empate.
- **Fila 9, el `store` con contención: sigue el empate.** Con p = 24, Autores 461 ns y Anderson 470. Con p = 56, Autores 1 531, Folly 1 534 y Anderson 1 554. Son diferencias de un 2 % como mucho.
- **Fila 11, `store & reclaim` más rápido con contención: no se confirma.** Con Docker, Autores bajaba de 1 516 a 813 ns con p = 56. Aquí baja de 1 531 a 1 480, un 3 %, y con p = 24, de 461 a 456. Anderson y Libstdc++ se mueven igual de poco, entre −4 % y +9 %. Es decir, el efecto que describe el paper aquí no se distingue del ruido. Las 5 rondas de Docker de ese punto iban de 440 a 1 470 ns, así que su mediana de 813 pudo depender de la suerte de las rondas. Es una sospecha, no una medida.
- **Fila 12, la latencia de Williams: sigue «en parte», algo más débil.** Sin contención sigue siendo el más lento en las tres operaciones (32 a 45 ns). Con contención supera a Libstdc++ en 5 de 6 casos (con Docker, en los 6) y a Folly en 2 de 6 (con Docker, en 3). El caso que pierde frente a Libstdc++, el `load` con p = 24 (6 106 frente a 3 882 ns), tampoco es claro: los dos tienen rondas por debajo de 4,3 µs y rondas por encima de 12 µs.
- **Filas 8 y 10 también se repiten.** El `store` de Autores con p = 1 vuelve a ser el cuarto de cinco (22 ns, frente a 17 de Folly), y `store & reclaim` vuelve a ser más lento que `store` en los cinco.

Una advertencia: aquí cambian dos cosas a la vez, Docker y el número de rondas. Como Docker no cambia el rendimiento de estos programas (Parte 6.6), lo más probable es que las

diferencias con la tanda de Docker vengan del ruido entre rondas. Es una deducción.

6.3 Memoria: ¿quién deja basura?

Esta prueba **no está en el paper**: es `herramientas/bench_basura.cpp`. La mitad de los hilos escribe y la otra mitad lee, sin parar, y se cuentan los bloques de memoria vivos en cada instante. La raya discontinua es el máximo posible si todo se liberara en el acto.



Pico de bloques de memoria vivos

Hilos	Límite	Autores	Anderson	Folly	Williams	Libstdc++
2	12	9	**4 013**	9	9	9
4	22	13	**6 808**	14	15	13
8	42	21	**9 910**	23	25	21
16	82	36	**17 079**	39	43	36
24	122	52	**19 960**	53	59	53

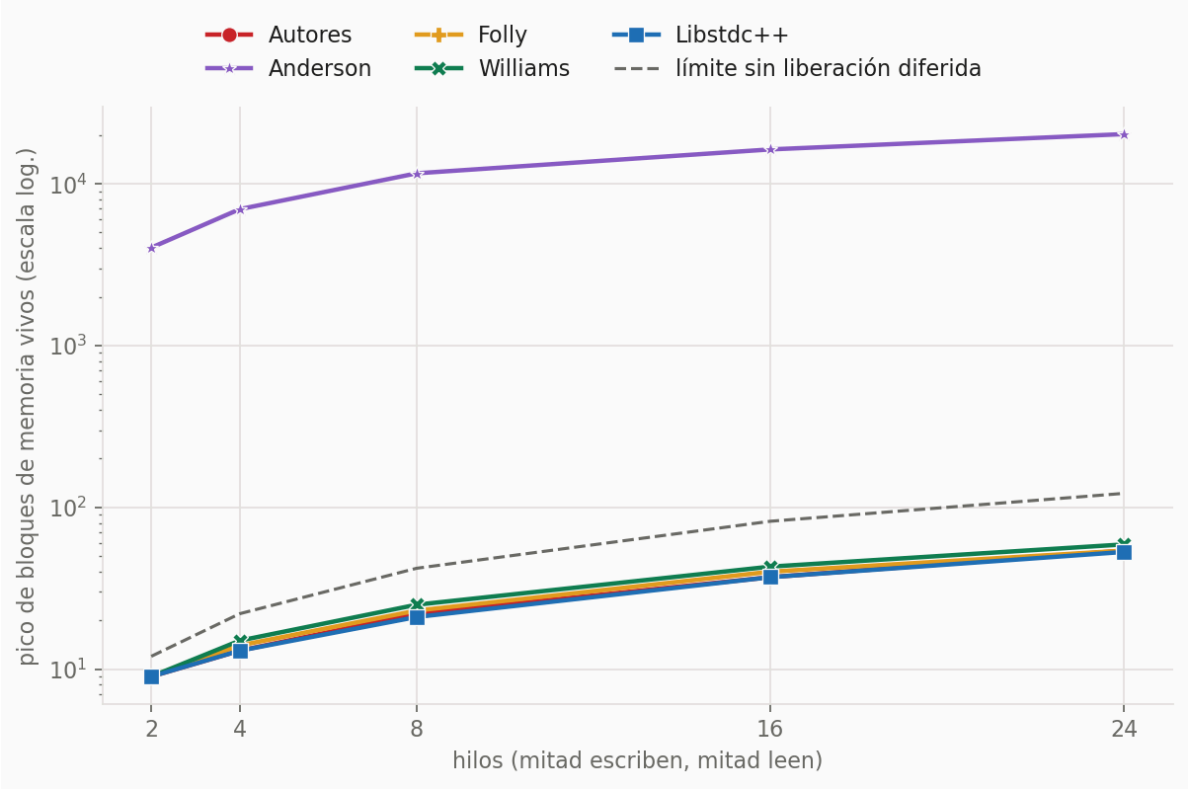
Abajo hay cuatro líneas casi pegadas (Autores queda tapada por Libstdc++), siempre bajo el límite. Liberan en el mismo instante en que el último usuario termina. Arriba está Anderson, con unas 380 veces más bloques cuando hay 24 hilos: su caja de pendientes.

Antes de condenar a Anderson, tres matices:

- **Lo que espera en la caja es solo el bloque de control, la libreta.** El objeto muere en el acto: su pico de objetos vivos es 14, igual que los demás y bajo el límite de 37. Lo vimos en el código, y esta prueba lo confirma.
- **En bytes es poco.** Esos 19 800 bloques de más son 469 KB, frente a 2,3 KB de Autores. Es 200 veces más, pero sigue siendo menos de medio mega.
- **Por eso la memoria total del proceso no lo delata.** Con n = 10^6 y 24 hilos, Anderson usa 910 MB y Autores 1 074 MB, porque ahí lo que pesa son los datos. Williams ocupa 1

667 MB; su celda es el doble de grande, aunque no se comprobó si esa es toda la causa.

¿Y sin Docker? Sale lo mismo. Con los mismos programas oficiales, fuera de Docker, Anderson llega a 20 256 bloques con 24 hilos, unos 476 KB, y los demás se quedan entre 53 y 59, bajo el límite de 122. Su pico de objetos vivos vuelve a ser 14. Los bloques están en la tabla «Memoria retenida (nativo, binarios oficiales)» de `tablas.md`; los bytes y los objetos, en `nativo_oficial/basura.csv`. La gráfica es casi idéntica:



Pico de bloques de memoria vivos, sin Docker

El paper dice que Anderson tiene un «pequeño sobrecoste de espacio», y es exacto: pequeño sí, cero no. Autores sí consigue el cero. Dicho en una frase: Anderson compra velocidad con una caja de pendientes pequeña; Autores no necesita caja.